

Assignment 5 Report

CS334: Operating Systems Lab

Doubly indirect block and soft links

Team Members:

Maddala Bogadi Sushanth (Roll No. 230101061)
Varshith Grandhi (Roll No. 230101041)
Akshay Bobbala (Roll No. 230101009)
Muduguru Mahedhar Reddy (Roll No. 230101067)

Drive link.

Contents

1	Task- 5.1: Doubly-Indirect Block Support	3
1.1	Introduction	3
1.2	Implementation Methodology	3
1.3	Implementation	3
1.3.1	Modification in param.h	3
1.3.2	Modification in fs.h	4
1.3.3	Modification in file.h and fs.h regarding <code>addrs[]</code>	4
1.3.4	Modification in <code>bmap()</code> and <code>itrunc()</code> in <code>fs.c</code> file	4
1.4	Testing and Results	5
2	Task 5.2: Symbolic Links (Soft Links)	7
2.1	Introduction	7
2.2	System call : SYMLINK	7
2.3	Explanation of the symlink mechanism	8
2.3.1	New file type added	8
2.3.2	Modified <code>open()</code> behavior	8
2.3.3	<code>O_NOFOLLOW</code> FLAG	8
2.4	Recursive resolution handling and depth limit	8
2.4.1	Recursive resolution	8
2.4.2	Depth limit mechanism	8
2.5	Testing and Results	9
2.5.1	Difficulties Faced	10
2.5.2	Conclusions	10

1 Task- 5.1: Doubly-Indirect Block Support

1.1 Introduction

We had modified the xv6 file system to support larger files using a doubly-indirect block pointer. This extends the maximum file size beyond what is currently supported by xv6

The xv6 file system supports 12 direct data block addresses and one single indirect address per inode. This limits the maximum file size to approximately 268 KB. We had extended the file system to allow much larger files by adding a doubly-indirect pointer.

Initial number of blocks supported by xv6

12 direct accesses + 1 single level indirect access, we get $12+256$ blocks i.e, approx 268KB

Modified number of blocks supported by xv6

11 direct accesses + 1 single level indirect access + 1 double level indirect access, we get $12+256+256*256 = 65803$ blocks i.e, approx 64MB.

1.2 Implementation

Methodology

We had added a new structure that supports doubly indirect block accessing.
We implemented the logic for allotting and freeing blocks in the two level indirection way.

1.3 Implementation

1.3.1 Modification in param.h

FSSIZE represents the maximum number of blocks a file can hold in file system
As now we are going to increase the maximum supported file size, we should increase

FSSIZE to a quantity greater than 65803

We had modified the **FSSIZE** macro in param.h to **100000** from 2000.

1.3.2 Modification

in

fs.h

The new inode structure should still fit within the 128-byte size limit, so we have decreased the number of **NDIRECT** from 12 to 11 and added two new macros namely **MAXFILE** and **DOUBLY_INDIRECT**.

NDOUBLY_INDIRECT = **NINDIRECT** * **NINDIRECT**

MAXFILE = **NDIRECT** + **NINDIRECT** + **NDOUBLY_INDIRECT**

1.3.3 Modification in file.h and fs.h regarding `addrs[]`

In file.h, we had changed the size of `addrs[]` array of structure `inode` from `NDIRECT+1` to `NDIRECT+2` (`addrs[NDIRECT]` will have the indirect block and `addrs[NDIRECT+1]` will have the doubly-indirect block). This is the in-memory copy of the inode structure. Similarly, we had changed in fs.h, the on-disk copy by updating the size of `addrs[]` array to `NDIRECT+2`

```
#define NDIRECT 11 // task 1
#define NINDIRECT (BSIZE / sizeof(uint))
#define NDOUBLY_INDIRECT (NINDIRECT*NINDIRECT)
#define MAXFILE (NDIRECT + NINDIRECT + NDOUBLY_INDIRECT)

// On-disk inode structure
struct dinode {
    short type; // File type
    short major; // Major device number (T_DEVICE only)
    short minor; // Minor device number (T_DEVICE only)
    short nlink; // Number of links to inode in file system
    uint size; // Size of file (bytes)
    uint addrs[NDIRECT+2]; // Data block addresses
};
```

Figure 1.1: Modification in fs.h

```
// in-memory copy of an inode
struct inode {
    uint dev; // Device number
    uint inum; // Inode number
    int ref; // Reference count
    struct sleeplock lock; // protects everything below here
    int valid; // inode has been read from disk?

    short type; // copy of disk inode
    short major;
    short minor;
    short nlink;
    uint size;
    uint addrs[NDIRECT+2]; // Task 5.1: NDIRECT, NINDIRECT, NDOUBLY_INDIRECT
};
```

Figure 1.2: Modification in file.h

1.3.4 Modification in `bmap()` and `itrunc()` in fs.c file

To support the doubly-indirect addressing:

1. Change the `bmap()` function to allocate the blocks for doubly indirection.
2. While deleting the file, `itrunc()` is called and the logic is modified so that both the level of indirection pointers are freed.

- Doubly-indirect block contains 256 pointers to indirect blocks, each of which points to 256 data blocks.
- If the logical block number exceeds the single indirect range, traverse the doubly-indirect pointer and we can allocate if it is in the range.

```
// in-memory copy of an inode
struct inode {
    uint dev; // Device number
    uint inum; // Inode number
    int ref; // Reference count
    struct sleeplock lock; // protects everything below here
    int valid; // inode has been read from disk?

    short type; // copy of disk inode
    short major;
    short minor;
    short nlink;
    uint size;
    uint addrs[NDIRECT+2]; // Task 5.1: NDIRECT, NINDIRECT, NDOUBLY_INDIRECT
};
```

Figure 1.3: file.h

- When we have to go through the indirect blocks, first we should pass to the 1st level of indirection and again we should go to the address pointed by the value at that level to go to the actual block.

```

bn -= NINDIRECT;
if (bn < NDOUBLY_INDIRECT) {
    if ((addr = ip->addrs[NINDIRECT+1]) == 0){
        addr = balloc(ip->dev);
        if (addr == 0)
            return 0;
        ip->addrs[NINDIRECT+1] = addr;
    }
    bp = bread(ip->dev, ip->addrs[NINDIRECT+1]);
    a = (uint*)bp->data;

    // now, go to the second layer
    uint outer_index = bn / NINDIRECT;

    if ((addr = a[outer_index]) == 0){
        addr = balloc(ip->dev);
        if (addr == 0){
            brelse(bp);
            return 0;
        }
        a[outer_index] = addr;
        log_write(bp);
    }
    brelse(bp);

    // the final page from disk
    uint inner_index = bn % NINDIRECT;

    bp = bread(ip->dev, addr);
    a = (uint*)bp->data;

    if ((addr = a[inner_index]) == 0){
        addr = balloc(ip->dev);
        if (addr == 0){
            brelse(bp);
            return 0;
        }
        a[inner_index] = addr;
        log_write(bp);
    }
    brelse(bp);
    return addr;
}
panic("bmap: out of range");
}

```

Figure 1.4: Modification in bmap()

```

// Task 5.1: free the doubly indirect blocks
if (ip->addrs[NINDIRECT + 1]){
    bp = bread(ip->dev, ip->addrs[NINDIRECT+1]);
    a = (uint*)bp->data;
    for(j = 0; j < NINDIRECT; j++){
        if (a[j]){
            bp2 = bread(ip->dev, a[j]);
            a2 = (uint*)bp2->data;
            for(k = 0; k < NINDIRECT; k++){
                if (a2[k])
                    bfree(ip->dev, a2[k]);
            }
            brelse(bp2);
            bfree(ip->dev, a[j]);
        }
    }
    brelse(bp);
    bfree(ip->dev, ip->addrs[NINDIRECT+1]);
    ip->addrs[NINDIRECT+1] = 0;
}

ip->size = 0;
iupdate(ip);
}

```

Figure 1.5: Modification in itrunc()

1.4 Testing and Results

A test program `bigfile.c` was written to validate the implementation.

The output clearly demonstrates correct functioning: supports large files that require doubly indirect block addressing, we can see that over 9000 blocks are created for a file.

The default xv6 can't support a file with more than 268 blocks.

```

19 int
20 main(int argc, char *argv[])
21 {
22     int fd;
23     char *name = "bigfile";
24     char buf[BS];
25     int i;
26     // blocks to write: choose <= theoretical limit (65536). 50000 is safe for BS=1024
27     int blocks = 10000;
28
29     for(i = 0; i < BS; i++) buf[i] = (char)(i & 0xFF);
30
31     // remove existing file if any (optional)
32     unlink(name);
33
34     fd = open(name, O_CREATE | O_WRONLY);
35     if(fd < 0){
36         printf("open failed\n");
37         exit(1);
38     }
39
40     for(i = 0; i < blocks; i++){
41         if(i%500==0) printf("write at block %d\n", i);
42         if(write(fd, buf, BS) != BS){
43             printf("write failed at block %d\n", i);
44             close(fd);
45             exit(1);
46         }
47     }
48     close(fd);
49     printf("done with write\n");
50
51     // Reopen and verify
52     fd = open(name, O_RDONLY);
53     if(fd < 0){
54         printf("open for read failed\n");
55         exit(1);
56     }
57     for(i = 0; i < blocks; i++){
58         char rbuf[BS];
59         if(read(fd, rbuf, BS) != BS){
60             printf("read failed at block %d\n", i);
61             close(fd);
62             exit(1);
63         }
64         if(memcmp(rbuf, buf, BS) != 0){
65             printf("data mismatch at block %d\n", i);
66             close(fd);
67             exit(1);
68         }
69     }
70     close(fd);
71     printf("bigfile test passed: %d blocks (%d bytes) written and verified\n", blocks, blocks*BS);
72     exit(0);
73 }

```

Figure 1.6: bigfile.c

```

hart 2 starting
hart 1 starting
init: starting sh
$ bigfile
write at block 0
write at block 500
write at block 1000
write at block 1500
write at block 2000
write at block 2500
write at block 3000
write at block 3500
write at block 4000
write at block 4500
write at block 5000
write at block 5500
write at block 6000
write at block 6500
write at block 7000
write at block 7500
write at block 8000
write at block 8500
write at block 9000
write at block 9500
done with write
bigfile test passed: 10000 blocks (10240000 bytes) written and verified

```

Figure 1.7: Output of bigfile.c

2 Task 5.2: Symbolic Links (Soft Links)

2.1 Introduction

In this task, we had extended the xv6 file system to support symbolic (soft) links.

Symbolic links are special files that point to other files or directories. Symbolic links refer to a linked file by pathname; when a symbolic link is opened, the kernel follows the link to the referred file. Symbolic links resemble hard links, but hard links are restricted to pointing to file on the same disk, while symbolic links can cross disk devices.

2.2 System call : SYMLINK

We had implemented a new system call `symlink(target, path)` that creates a symbolic link at `path` referring to `target`. When opening a symbolic link, xv6 resolves the link and opens the target file transparently.

- Creates a new inode of type `T_SYMLINK`
- Stores the target path as the link's content using `writei()`
- The inode itself does not contain actual file data, only the path to the target

```
uint64
sys_symlink(void)
{
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;

    if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
        return -1;

    begin_op();
    if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
        end_op();
        return -1;
    }

    // Write target path into the symlink's data block
    writei(ip, 0, (uint64)target, 0, strlen(target) + 1);
    iunlockput(ip);
    end_op();

    return 0;
}
```

Figure 2.1: Initialization

2.3 Explanation of the symlink mechanism

2.3.1 New file type added

We introduced a new file type `T_SYMLINK` in `stat.h` to distinguish symbolic links from regular files and directories.

2.3.2 Modified `open()` behavior

1. When `sys_ open()` encounters a file whose inode type is `T_SYMLINK`, it reads the stored target path from the file's data block
2. It then performs a `namei()` lookup on this path to find the real target inode.
3. Once resolved, the system continues as if the target file was opened directly.
4. This ensures transparency — user programs can read or write through a symlink without extra steps.

2.3.3 `O_NOFOLLOW` FLAG

1. If a user specifies the `O_NOFOLLOW` flag when opening a symbolic link, `xv6` will open the link file itself rather than following it
2. This behavior matches standard UNIX semantics and is useful for operations like inspecting or deleting the link.

2.4 Recursive resolution handling and depth limit

2.4.1 Recursive resolution

Each time `open()` finds a `T_SYMLINK`, it:

- Reads the target path from the link's data block.
- Performs `namei(target)` to locate the next inode.
- Locks and checks the new inode's type.

This loop continues until a regular file or directory is reached.

2.4.2 Depth limit mechanism

- A counter tracks the number of consecutive symlink resolutions.
- If the counter exceeds 10 levels, the kernel aborts the operation and returns an error (-1).
- This prevents infinite recursion caused by cyclic links (ex. $A \rightarrow B \rightarrow A$).


```

unlock(ip);
// task 5.4: resolve symlinks
for (int depth = 0; ip->type == T_SYMLINK && !(omode & O_NOFOLLOW); depth++) {
    if (depth >= 10) {
        iunlockput(ip);
        end_op();
        return -1;
    }

    char target[MAXPATH];
    if (readi(ip, 0, (uint64)target, 0, MAXPATH) < 0) {
        iunlockput(ip);
        end_op();
        return -1;
    }
    iunlockput(ip);

    if((ip = namei(target)) == 0){
        end_op();
        return -1;
    }
}
iunlock(ip);

```

Figure 2.2: sys_open()

2.5 Testing and Results

A test program `symlinktest.c` was written to validate the implementation.

The output clearly demonstrates required functioning:

- Created symbolic links to files and directories
- Tested recursive links and detect loops.
- Tested `O_NOFOLLOW` behavior

```

void
test_basic()
{
    printf("=== Basic symlink test ===\n");

    int fd = open("base.txt", O_CREATE | O_RDWR);
    write(fd, "xv6 symlink demo\n", 17);
    close(fd);

    check(symlink("base.txt", "linkA") == 0, "symlink creation failed");

    char buf[BUFSZ] = {0};
    fd = open("linkA", O_RDONLY);
    check(fd >= 0, "open linkA failed");

    read(fd, buf, sizeof(buf));
    close(fd);

    printf("linkA -> contents: %s\n", buf);
    unlink("base.txt");
    unlink("linkA");
}

void
test_nested_links()
{
    printf("=== Nested symlink test ===\n");

    int fd = open("f1.txt", O_CREATE | O_RDWR);
    write(fd, "Deep link success!\n", 20);
    close(fd);

    check(symlink("f1.txt", "l1") == 0, "symlink l1 creation failed");
    check(symlink("l1", "l2") == 0, "symlink l2 creation failed");
    check(symlink("l2", "l3") == 0, "symlink l3 creation failed");

    char buf[BUFSZ] = {0};
    fd = open("l3", O_RDONLY);
    check(fd >= 0, "open l3 failed");

    read(fd, buf, sizeof(buf));
    close(fd);

    printf("Resolved through 3-level link: %s\n", buf);

    unlink("f1.txt");
    unlink("l1");
    unlink("l2");
    unlink("l3");
}

```

Figure 2.3: `symlinktest.c`

```

void
test_loop_detection()
{
    printf("=== Loop detection test ===\n");

    // create circular symlinks: A -> B, B -> A
    check(symlink("loopA", "loopB") == 0, "symlink loopA failed");
    check(symlink("loopB", "loopA") == 0, "symlink loopB failed");

    int fd = open("loopA", O_RDONLY);
    check(fd < 0, "loop detection failed (should not resolve infinitely)");

    printf("Loop correctly detected and blocked.\n");

    unlink("loopA");
    unlink("loopB");
}

void
test_on_directory()
{
    printf("=== Symlink to directory test ===\n");

    mkdir("dir1");
    check(symlink("dir1", "dirlink") == 0, "symlink to dir failed");

    int fd = open("dirlink", O_RDONLY);
    if (fd < 0) {
        printf("Symlink to directory cannot be opened as file (expected)\n");
    } else {
        printf("Unexpectedly opened symlinked directory\n");
        close(fd);
    }

    unlink("dirlink");
    unlink("dir1");
}

void
test_nofollow()
{
    printf("=== O_NOFOLLOW test ===\n");

    int fd = open("target.txt", O_CREATE | O_RDWR);
    write(fd, "no-follow", 9);
    close(fd);

    check(symlink("target.txt", "lnk") == 0, "symlink creation failed");

    int fd_link = open("lnk", O_RDONLY | O_NOFOLLOW);
    check(fd_link >= 0, "open with O_NOFOLLOW failed");

    struct stat st;
    fstat(fd_link, &st);
    check(st.type == T_SYMLINK, "O_NOFOLLOW opened target instead of link");

    printf("O_NOFOLLOW correctly opened symlink inode.\n");

    close(fd_link);
    unlink("lnk");
    unlink("target.txt");
}

```

Figure 2.4: `symlinktest.c`

```

xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$ symlinktest

== Starting symlink tests ==
Basic symlink test
linkA -> contents: xv6 symlink demo

Nested symlink test
Resolved through 3-level link: Deep link success!

Loop detection test
Loop correctly detected and blocked.
Symlink to directory test
Unexpectedly opened symlinked directory
O_NOFOLLOW test
O_NOFOLLOW correctly opened symlink inode.
== All symbolic link tests passed ==
$

```

Figure 2.5: Output of `symlinktest.c`

2.5.1 Difficulties Faced

- Managing two-level block allocation and traversal in `bmap()` was complex.
- Ensuring correct freeing of all blocks in `itrunc()` without corruption was challenging.

2.5.2 Conclusions

- Doubly indirect blocks increased the maximum file size significantly from 268KB to 64MB.
- Implemented symbolic links to point other files/directories.